ChatGPT

Trees for FAANG Interviews in JavaScript

Executive Summary

Trees are non-linear, hierarchical data structures used to model "contains/owns/parent-child" relationships, where each node can be the root of its own subtree—making recursion and divide-and-conquer especially natural. [1] In technical interviews at Google [2] and Meta [3], tree problems are common because they reliably test (1) decomposition into subproblems, (2) invariants, (3) edge-case discipline around `null`, and (4) time/space reasoning under traversal choices (DFS/BFS). [4]

Core idea that unlocks most tree questions: decide *what your helper returns* (boolean? height? best "downward" path? tuple?) and pick a traversal order that makes that return value easy to compute (preorder vs inorder vs postorder vs level-order). [5]

Must-memorize facts and patterns

These are the high-yield items that show up repeatedly across FAANG-style tree interviews:

1. **Tree terminology**: root, leaf, subtree, ancestor/descendant, path, depth vs height. Depth is typically edges from root; height is edges down to deepest leaf. [6]
2. **Traversal orders**:
3. **Preorder (NLR)**: "process node before children" (great for copy/transform like inversion). [7]
4. **Inorder (LNR)**: for BSTs, inorder yields sorted order (strictly increasing for a valid BST with strict inequalities). [8]
5. **Postorder (LRN)**: "compute from children up" (great for height/diameter/balance/max-path-sum). [7]
6. **Level-order (BFS)**: process level-by-level using a queue. [9]
7. **Time complexity baseline**: If you visit each node once, it's usually **O(n)** time, where *n = number of nodes*. [10]
8. **Space complexity discipline**:
9. DFS recursion uses **O(h)** call stack where *h = tree height*; worst-case skewed tree: **O(n)**. [6]
10. BFS queue can grow to **O(w)** where *w = max width*; worst-case **O(n)**. [11]
11. **JavaScript-specific risk**: deep recursion can throw "Maximum call stack size exceeded"; interviewers often ask for an iterative alternative using an explicit stack/queue. [12]
12. **BST invariants**: validating a BST requires *range propagation* (min/max bounds), not only local parent-child comparisons. [8]
13. **Postorder "return height" pattern**: any problem that asks about height, balance, diameter, or "best path" usually becomes "return something computed from children." [13]
14. **Level boundary capture in BFS**: for per-level outputs you must snapshot `levelSize = queue.length` before iterating, because the queue changes as you push children. [14]
15. **Two outputs in one traversal**: many problems require returning a value from recursion *and* updating a global/outer "best so far" (diameter, max path sum). [15]

16. **Serialization contract**: your serializer/deserializer must preserve shape; missing children must be encoded explicitly (e.g., sentinel tokens). [16]

Foundations: Trees & Binary Trees

Definitions you should be able to say out loud

A **tree (data structure)** is a hierarchical structure of nodes connected by edges, typically with a distinguished **root**; every node except the root has exactly one parent, and there are no cycles. [1] A **binary tree** is a tree where each node has at most two children (commonly called `left` and `right`). [1]

Key terms (interviewers use these precisely):

- **Node / edge**: nodes plus connections. [1]
- **Root**: topmost node; has no parent. [1]
- **Leaf**: node with no children. [6]
- **Subtree**: a node plus all its descendants. [17]
- **Path**: sequence of nodes/edges (often measured in edges or nodes depending on the problem). [18]
- **Depth vs height**: depth counts edges from root down to a node; height counts edges from a node down to deepest leaf. [19]
- **Ancestor / LCA**: ancestor is any node on the path to root; LCA is the lowest node that is an ancestor of both nodes. [20]

Traversals and why they matter

Every tree problem is "just a traversal + a state invariant."

- **DFS traversals** (depth-first): go deep, then backtrack. Common DFS orders are preorder/inorder/postorder. [21]
- **BFS traversal** (breadth-first): explore level-by-level using a queue. [22]

A good mental model: - If you need to *build/transform* top-down: **preorder**. - If you need to *compute something from children*: **postorder**. - If it's a BST and you want sorted order: **inorder**. - If you need "per level" logic: **BFS**. [23]

DFS vs BFS tradeoffs you must justify

- **DFS** is often simplest recursively and uses **O(h)** stack space, where `h` is height. Worst-case skewed: **O(n)**. [6]
- **BFS** uses an explicit queue and can take **O(n)** space in the worst case; it's natural when answers are defined per-level or by shortest distance in unweighted graphs. [11]
- **JavaScript follow-up**: for extremely deep trees, recursion may overflow the call stack; iterative DFS (explicit stack) is the safe alternative. [12]

Recursion mental model

A tree recursion should answer two questions:

1. **Base case**: what does `null` mean for this problem? (height 0? balanced true? valid BST true? etc.)
2. **Return-value meaning**: what does the helper "promise" to compute for the caller?

When this is correct, recursion becomes mechanically reliable because every node is treated as "root of a smaller problem," a property explicitly aligned with the subtree view of trees. [1]

Must-know checklist

- Be able to write DFS template (recursive and iterative).
- Be able to write BFS template with correct level boundaries.
- Always decide if path length is in **edges** or **nodes** (LeetCode varies by problem). [24]
- For BST validation, use **(min, max)** propagation (strict inequality!). [8]
- Avoid global state that isn't reset per call (common JS pitfall).
- Handle `null` root explicitly.
- Know that "balanced" means **every node** satisfies the height condition, not just the root. [25]
- Be ready to convert recursion → iterative when depth can be extreme (explicit stack frame or BFS). [12]

Warm-up Mini Problem

Mini problem

Binary Tree Preorder Traversal (Easy): return an array of values in preorder (Node → Left → Right). This is not in your NeetCode list, but it teaches the most reusable skill: turning a recursive visit order into an iterative explicit stack.

Preorder is a canonical DFS order. [7]

Example tree and order

Example:

```
      1
    /  \
  2     3
       /
      4
```

Preorder visitation order: `1, 2, 3, 4`

State idea: “process current node immediately, then schedule its children.”

JavaScript (recursive and iterative)

```
// LeetCode-style definition used throughout this paper.
class TreeNode {
  constructor(val = 0, left = null, right = null) {
    this.val = val;
    this.left = left;
    this.right = right;
  }
}

/**
 * Preorder traversal (recursive).
 * Invariant: preorder(node) returns preorder list for the subtree rooted at
node.
 */
function preorderTraversalRecursive(root) {
  const result = [];
  function dfs(node) {
    if (node === null) return;
    result.push(node.val);     // Node
    dfs(node.left);            // Left
    dfs(node.right);           // Right
  }
  dfs(root);
  return result;
}

/**
 * Preorder traversal (iterative, safe for deep trees).
 * Invariant: stack holds nodes whose "Node step" is still pending in correct
preorder order.
 */
function preorderTraversalIterative(root) {
  if (root === null) return [];
  const result = [];
  const stack = [root];

  // While there exists at least one node whose value has not been processed.
  while (stack.length > 0) {
    const node = stack.pop();
    result.push(node.val);

    // Push right first so that left is processed first (LIFO stack).
    if (node.right !== null) stack.push(node.right);
```

```
    if (node.left !== null) stack.push(node.left);
  }

  return result;
}
```

Why the while-loop condition is correct: `stack.length > 0` means there remains at least one node not yet visited in our explicit DFS schedule; when it reaches 0, all scheduled nodes have been processed.

Common Use Cases in Real Systems

Trees show up in production systems because "nested structure" is everywhere:

DOM and UI hierarchies

Browsers represent an HTML/XML document as a **DOM tree**, where each node represents part of the document. [26] Traversals here are often DFS-like (walk subtrees) for operations like searching, updating, or applying transformations.

In React [27], the UI is also modeled as trees built from components; React's docs explicitly discuss understanding your UI as a tree and how React creates a UI tree from components. [28]
- Typical traversal pattern: DFS over component subtree (conceptually) for rendering and reconciliation; in interviews, this maps to "walk a tree and compute/update derived data."

Filesystems and directory structures

A filesystem directory structure is naturally a tree: directories contain children (files/subdirectories). Searching for a file can be framed as DFS (deep search) or BFS (level-by-level). While implementations vary, the conceptual mapping is that BFS explores "all siblings before deeper descendants," and DFS explores "descendants first," mirroring standard traversal definitions. [29]

Organization charts and ownership graphs

Org charts (manager → reports) are tree-shaped; typical tasks are: - "find chain of command" (ancestor path), - "count reports" (subtree size), - "find common manager" (LCA variant).

Tries for routing, autocompletion, and prefix lookups

A trie (prefix tree) stores strings across a tree of characters and is widely used for prefix-based retrieval such as autocomplete. [30]

ASTs in compilers and linters

An AST is a tree representation of code structure used for analysis and transformation; each node represents a language construct. [31] Traversal pattern: almost always DFS (often postorder) to compute properties bottom-up.

"Tree modeling" in Laravel domain data

In Laravel [32], representing hierarchical data (categories/subcategories, threaded comments, org charts) is commonly done with a **self-referential** model: a row has a `parent_id` pointing to another row in the same table. This uses standard one-to-many and inverse/belongs-to relationships. [33]
- Typical traversal pattern: when you eager-load nested children recursively, you conceptually traverse a tree of models (DFS) to render nested menus or compute aggregates.

Repeated Concepts & Gotchas

The "what does recursion return?" trap

Many candidates can write recursion, but fail interviews because they return the wrong *meaning*:

- Return **boolean**: "Is this subtree valid/balanced/same?"
- Return **height**: "How tall is this subtree?"
- Return **tuple/object**: "(isBalanced, height)" or "(min, max, isBST)"
- Return **best downward score** while updating a global best: max path sum is the archetype. [34]

A high-signal interviewer follow-up is: "What does your helper return and why?"

Global-variable pitfalls in JS

Common failure mode: - Define `let ans = 0` outside the function and forget to reset it between test cases (or between runs). Keep globals *inside* the exported function.

Null handling

Every tree problem needs a decision: - `null` height: 0 (common), or -1 (some definitions). LeetCode variants typically align to problem statement. [35]
- `null` tree validity: usually true (empty tree is valid BST, empty subtree matches). [36]

Double-counting paths

Path problems often have two distinct notions: - "best downward path from node" (returned) - "best path anywhere in subtree" (global max updated)

If you mix these, you'll incorrectly allow "forking upward twice" (illegal) or drop valid through-node paths. [34]

BST invariants must be global, not local

Checking only `node.left.val < node.val < node.right.val` is insufficient because violations can occur deeper in the subtree. Correct approach: propagate allowed `(min, max)` bounds. [8]

Serialization pitfalls

If you serialize without recording `null` children, multiple different shapes can serialize to the same string; deserialization becomes ambiguous. [16]

Stack/queue memory growth

BFS can store many nodes in memory at once; DFS recursion can overflow JS call stack for deep trees, causing "too much recursion / Maximum call stack size exceeded." [37]

NeetCode 150 Tree Problems

The problem set below is commonly practiced through NeetCode [38] and LeetCode [39], which provide curated lists and canonical problem statements. [40]

Invert Binary Tree

A) Problem & Difficulty

Difficulty: Easy. [41]
Restatement: Given the root of a binary tree, swap every node's left and right child, returning the root of the inverted tree. [42]
What to notice immediately: - Every subtree is itself a binary tree; inversion is naturally recursive. [43]
- You can do this with preorder DFS (swap at node, then recursively invert children). [44]
- Iterative follow-up is straightforward (stack or queue), good for depth safety. [12]

B) Brute Force Idea (if applicable)

There isn't a meaningful "brute force" alternative; the direct traversal is already optimal since you must touch each node to swap pointers.

C) Optimal Strategy

Pattern: Preorder DFS (transform node, then descend). [44]
Invariant (one sentence): **After processing `node`, the subtree rooted at `node` is inverted, and recursion/stack ensures the same for all descendants.**

Key insight: - Swapping children at every node is independent; correctness composes across subtrees. [43]

D) Step-by-step Walkthrough (mandatory)

Example:

```
     4
    / \
   2   7
  / \ / \
 1  3 6  9
```

At node `4`: swap children → left becomes `7`, right becomes `2`.
Then recursively invert subtree rooted at `7` and subtree rooted at `2`.

Call stack sketch (recursive): - invert(4) - invert(7) - invert(9) → leaf - invert(6) → leaf - invert(2) - invert(3) → leaf - invert(1) → leaf

Each frame returns the same node reference after its children pointers are swapped.

E) JavaScript Solution (ES2020+)

```javascript
/**
 * Recursive (simple).
 * Invariant: invertTree(node) returns the root of the inverted subtree of node.
 */
function invertTree(root) {
  if (root === null) return null;

  // Swap children at this node.
  const tmp = root.left;
  root.left = root.right;
  root.right = tmp;

  // Recursively invert the (now swapped) children subtrees.
  invertTree(root.left);
  invertTree(root.right);

  return root;
}

/**
 * Iterative follow-up (depth-safe).
 * Invariant: stack holds nodes whose children still need to be swapped.
 */
function invertTreeIterative(root) {
  if (root === null) return null;
  const stack = [root];
```

```
  // While there exists at least one node not yet processed (not yet swapped).
  while (stack.length > 0) {
    const node = stack.pop();

    const tmp = node.left;
    node.left = node.right;
    node.right = tmp;

    if (node.left !== null) stack.push(node.left);
    if (node.right !== null) stack.push(node.right);
  }

  return root;
}
```

Why `while (stack.length > 0)` is correct: the stack is exactly the set of nodes still awaiting inversion; when empty, all nodes have had their children swapped.

F) Complexity

Time: O(n), each node swapped once. [42]
Space: - Recursive: O(h) call stack (can be O(n) if skewed). [45]
- Iterative: O(h) explicit stack average; worst-case O(n). [1]

G) Edge Cases & Gotchas - `root = null` → return null. - Single node → unchanged. - Skewed chain (all left or all right) → still works. - Large depth in JS recursion may overflow; use iterative. [12]
- Ensure you swap before descending (or consistently after; both work but must be coherent).

H) Common Wrong Approaches - Swapping only at the root (forgets subtrees): violates invariant "every node swapped." - Recursing into original pointers incorrectly (rare JS bug if you cache children and then swap inconsistently).

I) Interview Soundbite
"I do a DFS over the tree and at each node swap `left` and `right`. Because every subtree is itself a tree, recursively applying the same operation inverts the entire structure. If depth is a concern in JavaScript, I use an explicit stack to avoid call-stack overflow." [46]

J) Practice Checks 1) Q: What traversal order is most natural here and why?
A: Preorder, because we transform the node before needing any child-derived result. [47]
2) Q: If you do postorder instead, is it still correct?
A: Yes—swapping children commutes with subtree inversion, but preorder is simpler to explain.
3) Q: Why might iterative be preferred in JS?
A: Deep recursion can throw "Maximum call stack size exceeded." [12]

Gotcha test case: - Input: `[1,2,null,3,null,4]` (deep left chain)
- Expected output after invert: `[1,null,2,null,3,null,4]`

Maximum Depth of Binary Tree

A) Problem & Difficulty

Difficulty: Easy. [48]
Restatement: Return the maximum depth (number of nodes along the longest root-to-leaf path). [48]
What to notice immediately: - The phrase “longest root-to-leaf path” screams “height/depth recursion.” [49]
- Two clean solutions: DFS recursion (postorder) or BFS level counting. [50]
- BFS is naturally depth-safe in JS (no recursion). [12]

B) Brute Force Idea (if applicable)

Not really; computing depth requires visiting nodes. The DFS/BFS solutions are already optimal.

C) Optimal Strategy

Pattern: Postorder DFS returning height. [51]
Invariant: `dfs(node)` **returns the maximum depth of the subtree rooted at** `node`**.**

Key insight: - Depth at a node is `1 + max(leftDepth, rightDepth)`. [52]

D) Step-by-step Walkthrough (mandatory)

Example:

```
      3
     / \
    9   20
       / \
      15   7
```

Compute bottom-up: - `dfs(9) = 1` - `dfs(15) = 1`, `dfs(7) = 1` - `dfs(20) = 1 + max(1,1) = 2` - `dfs(3) = 1 + max(1,2) = 3`

Call stack (returns): - dfs(3) waits for dfs(9)=1 and dfs(20)=2 → returns 3

E) JavaScript Solution (ES2020+)

```javascript
/**
 * Recursive.
 * Invariant: dfs(node) returns max depth (in nodes) of node's subtree.
 */
function maxDepth(root) {
  function dfs(node) {
```

```
    if (node === null) return 0;
    const left = dfs(node.left);
    const right = dfs(node.right);
    return 1 + Math.max(left, right);
  }
  return dfs(root);
}

/**
 * Iterative BFS follow-up (depth-safe).
 * Invariant: queue holds all nodes in the current frontier (current level).
 */
function maxDepthBFS(root) {
  if (root === null) return 0;
  const queue = [root];
  let depth = 0;

  // While there exists at least one node at some unprocessed level.
  while (queue.length > 0) {
    const levelSize = queue.length; // snapshot boundary for this level
    depth += 1;

    for (let i = 0; i < levelSize; i++) {
      const node = queue.shift(); // OK for interview; you can optimize with
head index
      if (node.left !== null) queue.push(node.left);
      if (node.right !== null) queue.push(node.right);
    }
  }

  return depth;
}
```

Why `while (queue.length > 0)` is correct: if the queue is empty, there are no remaining nodes at any deeper level, so all levels have been processed. [22]

F) Complexity

Time: O(n). [48]
Space: - DFS recursion: O(h). [45]
- BFS queue: O(w) up to O(n). [11]

G) Edge Cases & Gotchas - `null` root → 0. - Single node → 1. - Skewed tree depth = n (DFS recursion risky in JS). [12]
- Confirm definition: LeetCode counts nodes, not edges. [48]
- BFS: must snapshot `levelSize` before processing a level. [53]

H) Common Wrong Approaches - Mixing "height in edges" with "depth in nodes," returning off-by-one. - BFS without snapshotting `levelSize` → levels bleed into each other. [54]

I) Interview Soundbite
"I compute depth bottom-up: depth at a node is one plus the max of its children's depths, with `null` returning 0. That's O(n) time. If recursion depth is a concern in JS, I do BFS and count levels explicitly."

J) Practice Checks 1) Q: What should `dfs(null)` return and why?
A: 0, because an empty subtree contributes no nodes to path length. [48]
2) Q: Why is this postorder-ish even though we don't say 'postorder'?
A: We need children results before computing parent depth. [47]
3) Q: What's BFS depth equal to?
A: The number of levels processed. [55]

Gotcha test case: - Input: `[]`
- Expected: `0`

Diameter of Binary Tree

A) Problem & Difficulty

Difficulty: Easy (conceptually), but requires "two outputs" mental model. [56]
Restatement: Return the diameter: the length of the longest path between any two nodes, measured in edges (LeetCode's definition for this problem). [57]
What to notice immediately: - Longest path may or may not pass through root. [57]
- The diameter through a node is `leftHeight + rightHeight` (in edges if height is edges). [58]
- Return value and global update: you typically return height, while updating best diameter. [15]

B) Brute Force Idea (if applicable)

Brute force: for each node, compute height of left and right (each height is O(n)), update diameter; total O(n^2) in worst case skewed trees. This is why a single postorder traversal is preferred. [59]

C) Optimal Strategy

Pattern: Postorder DFS returning height + updating global diameter. [60]
Invariant: `dfs(node)` **returns the height (in edges) of** `node`**'s subtree, and** `best` **is the maximum diameter seen so far over processed nodes.**

Key insight: - At node, the best path "through" it uses the deepest edge-path down left plus deepest edge-path down right. [61]

D) Step-by-step Walkthrough (mandatory)

Example:

```
      1
     / \
    2   3
   / \
  4   5
```

Heights in edges: - leaf (4): height 0 - leaf (5): height 0 - node 2: leftHeight=0, rightHeight=0 → through=0+0+2? Careful: if height is edges from node to leaf, then: - At node 2, height = 1 + max(0,0) = 1 - Diameter through node 2 in edges = (leftHeight + 1) + (rightHeight + 1) = 2 A simpler convention: - Let `dfs(node)` return height in **nodes**, then diameter in edges is `(leftNodes + rightNodes - 2)`; more error-prone. So we'll use this clean convention: - `dfs(null) = -1` height-in-edges - `dfs(leaf) = 0`

Then at node 2: - left = 0, right = 0 - through = left + right + 2 = 2 edges (4–2–5)

At node 1: - left height = 1 (1–2–4 edges: actually height from node 1 to deepest leaf is 2 edges; computed as 1 + max(0,0) = 1 for node 2, then node 1 is 1 + max(1,0) = 2) - right height = 0 - through = 2 + 0 + 2 = 4? That suggests mismatch; let's compute carefully with the `-1` base: - dfs(4)=0, dfs(5)=0 - dfs(2)=1 + max(0,0)=1 - dfs(3)=0 - dfs(1)=1 + max(1,0)=2 - diameter through 1 = leftHeight + rightHeight + 2 = 1 + 0 + 2 = 3 edges, which matches path 4–2–1–3 (3 edges). Global best becomes 3.

E) JavaScript Solution (ES2020+)

```javascript
/**
 * Recursive (postorder).
 * Invariant: dfs(node) returns height in edges; best tracks maximum diameter
(edges).
 */
function diameterOfBinaryTree(root) {
  let best = 0;

  function dfs(node) {
    if (node === null) return -1; // so leaf returns 0
    const leftH = dfs(node.left);
    const rightH = dfs(node.right);

    // The longest path that goes through this node uses the best downward edge-
paths.
    const through = leftH + rightH + 2;
    if (through > best) best = through;

    // Height in edges from this node to its deepest leaf.
    return 1 + Math.max(leftH, rightH);
  }

  dfs(root);
```

```
  return best;
}

/**
 * Iterative follow-up (depth-safe): postorder using explicit stack + Map for
heights.
 * Invariant: when a node is popped as "visited", children's heights are already
known.
 */
function diameterOfBinaryTreeIterative(root) {
  if (root === null) return 0;
  let best = 0;
  const height = new Map(); // node -> height in edges
  const stack = [[root, false]];

  while (stack.length > 0) {
    const [node, visited] = stack.pop();
    if (node === null) continue;

    if (!visited) {
      // Postorder: process children before node.
      stack.push([node, true]);
      stack.push([node.right, false]);
      stack.push([node.left, false]);
    } else {
      const leftH = node.left ? height.get(node.left) : -1;
      const rightH = node.right ? height.get(node.right) : -1;
      best = Math.max(best, leftH + rightH + 2);
      height.set(node, 1 + Math.max(leftH, rightH));
    }
  }

  return best;
}
```

Why the while-loop condition is correct: the stack represents scheduled frames of work; if it's empty, all nodes have been processed in postorder, so `best` is final.

F) Complexity

Time: O(n). [57]
Space: - Recursive: O(h). [45]
- Iterative: O(n) due to Map + stack in worst case.

G) Edge Cases & Gotchas - Empty tree → 0. - Single node → 0 (no edges). - Two nodes → 1. - Ensure you return height but update diameter separately. - Don't confuse height base case: `-1` makes leaf height `0` in edges.

H) Common Wrong Approaches - Returning diameter instead of height from DFS: caller can't combine results properly. - Off-by-one from mixing "nodes" vs "edges" definitions. [24]

I) Interview Soundbite
"I do a postorder DFS that returns the height of each subtree. At each node I compute the best diameter passing through it as `leftHeight + rightHeight + 2` and update a global maximum. One traversal gives O(n) time."

J) Practice Checks 1) Q: Why is postorder the natural traversal here?
A: A node's diameter-through value depends on children heights, so children must be computed first. [47]
2) Q: Why do we need a global `best`?
A: The max path can be entirely inside one subtree; it isn't always tied to the height returned to the parent. [56]
3) Q: What does `dfs(null) = -1` buy you?
A: It makes leaf height 0 in edges and makes through-formula consistent.

Gotcha test case: - Input: `[1,2]`
- Expected: `1`

Balanced Binary Tree

A) Problem & Difficulty

Difficulty: Easy. [25]
Restatement: Return true if the tree is height-balanced: for every node, left and right subtree heights differ by at most 1. [62]
What to notice immediately: - Balance must hold at **every** node (not just root). [62]
- Naively computing height for each node can be O(n^2).
- Best solution: postorder DFS returning height or a sentinel indicating "unbalanced." [62]

B) Brute Force Idea (if applicable)

Brute force: for each node, compute `height(left)` and `height(right)` (O(n) each), check difference, recurse; worst-case O(n^2). This fails on skewed trees.

C) Optimal Strategy

Pattern: Postorder DFS returning height; use sentinel `-1` to mean "unbalanced." [62]
Invariant: `dfs(node)` **returns the height of** `node` **if subtree is balanced, otherwise returns** `-1` **to propagate failure upward.**

Key insight: - The moment a subtree is unbalanced, you can short-circuit and bubble up `-1`.

D) Step-by-step Walkthrough (mandatory)

Example unbalanced:

```
     1
    /
   2
  /
 3
```

- dfs(3) = 1 (height in nodes convention here)
- dfs(2): left=1, right=0, diff=1 → balanced → height=2
- dfs(1): left=2, right=0, diff=2 → unbalanced → return -1
  Final: `dfs(root) !== -1` is false.

E) JavaScript Solution (ES2020+)

```
/**
 * Recursive.
 * Invariant: dfs(node) returns height (in nodes) if balanced; else -1.
 */
function isBalanced(root) {
  function dfs(node) {
    if (node === null) return 0;

    const leftH = dfs(node.left);
    if (leftH === -1) return -1;

    const rightH = dfs(node.right);
    if (rightH === -1) return -1;

    if (Math.abs(leftH - rightH) > 1) return -1;
    return 1 + Math.max(leftH, rightH);
  }

  return dfs(root) !== -1;
}

/**
 * Iterative follow-up (depth-safe): postorder with visited flag + Map heights.
 * Invariant: when visited=true, height of children already computed.
 */
function isBalancedIterative(root) {
  if (root === null) return true;

  const height = new Map();
  const stack = [[root, false]];
  let ok = true;
```

```
  while (stack.length > 0 && ok) {
    const [node, visited] = stack.pop();
    if (node === null) continue;

    if (!visited) {
      stack.push([node, true]);
      stack.push([node.right, false]);
      stack.push([node.left, false]);
    } else {
      const leftH = node.left ? height.get(node.left) : 0;
      const rightH = node.right ? height.get(node.right) : 0;
      if (Math.abs(leftH - rightH) > 1) ok = false;
      height.set(node, 1 + Math.max(leftH, rightH));
    }
  }

  return ok;
}
```

F) Complexity

Time: O(n). [62]
Space: - Recursive: O(h), risky for deep trees in JS. [12]
- Iterative: O(n) for Map + stack worst case.

G) Edge Cases & Gotchas - Empty tree → true. - Single node → true. - Skewed tree of length 3+ → false. - Make sure you check balance at *every* node. [25]
- Decide height convention (nodes vs edges) and stay consistent.

H) Common Wrong Approaches - Checking only root height difference (misses deeper imbalance). - Computing heights separately from balance and re-walking subtrees (O(n^2)).

I) Interview Soundbite
"I use a bottom-up DFS: each call returns the height if balanced, otherwise a sentinel -1. If any subtree returns -1, I propagate it upward immediately. This checks every node in one traversal, O(n)."

J) Practice Checks 1) Q: Why is top-down 'check heights then recurse' potentially O(n^2)?
A: Heights are recomputed for overlapping subtrees repeatedly.
2) Q: Why does returning -1 help?
A: It short-circuits once imbalance is found, keeping one-pass behavior.
3) Q: What's the iterative equivalent?
A: Postorder via stack + visited flag so child heights exist before parent computation.

Gotcha test case: - Input: `[1,2,2,3,3,null,null,4,4]`
- Expected: `false` [25]

Same Tree

A) Problem & Difficulty

Difficulty: Easy. [63]
Restatement: Given roots `p` and `q`, return true if they are structurally identical and all corresponding values match. [63]
What to notice immediately: - It's a synchronized traversal. - Base cases are the whole game: `null/null` is true; `null/non-null` is false. [64]

B) Brute Force Idea (if applicable)

Serializing both trees then comparing strings works but is unnecessary overhead and introduces serialization pitfalls.

C) Optimal Strategy

Pattern: DFS in lockstep. [64]
Invariant: `dfs(a,b)` **returns true iff the subtrees rooted at** `a` **and** `b` **are identical in structure and values.**

D) Step-by-step Walkthrough (mandatory)

Example: - If both nodes are null: identical at this position. - If one null: structure differs. - Else values must match and both pairs of children must match.

Call stack: `dfs(p,q)` fans to `(p.left,q.left)` and `(p.right,q.right)`.

E) JavaScript Solution (ES2020+)

```
/**
 * Recursive.
 * Invariant: same(a, b) is true iff subtrees at a and b are identical.
 */
function isSameTree(p, q) {
  if (p === null && q === null) return true;
  if (p === null || q === null) return false;
  if (p.val !== q.val) return false;
  return isSameTree(p.left, q.left) && isSameTree(p.right, q.right);
}

/**
 * Iterative follow-up: queue of node pairs.
 * Invariant: queue holds pairs that must match for trees to be identical.
 */
function isSameTreeIterative(p, q) {
```

```
  const queue = [[p, q]];

  while (queue.length > 0) {
    const [a, b] = queue.shift();

    if (a === null && b === null) continue;
    if (a === null || b === null) return false;
    if (a.val !== b.val) return false;

    queue.push([a.left, b.left]);
    queue.push([a.right, b.right]);
  }

  return true;
}
```

F) Complexity

Time: O(n) where n is the number of nodes visited (bounded by total size). [63]
Space: O(h) recursion or O(w) queue. [65]

G) Edge Cases & Gotchas - Both null → true. - One null → false. - Duplicate values but different structure → false. - Very deep trees: iterative avoids recursion overflow. [12]
- Ensure you check value equality *before* recursing.

H) Common Wrong Approaches - Compare inorder traversals only: different shaped trees can have same inorder. - Forgetting to compare structure (null-ness).

I) Interview Soundbite
"I traverse both trees at the same time. At each step, if both nodes are null I continue, if only one is null it's not the same, and if values differ it's not the same. Otherwise I recurse on left and right children."

J) Practice Checks 1) Q: Why can't you just compare sorted values?
A: Structure matters; same multiset of values can form many different trees.
2) Q: What's the most important base case?
A: `null/null` is true; `null/non-null` is false.
3) Q: BFS vs DFS—does it matter?
A: Either works; it's "pairwise traversal," not the order, that defines correctness.

Gotcha test case: - p = `[1,2]`, q = `[1,null,2]`
- Expected: `false`

Subtree of Another Tree

A) Problem & Difficulty

Difficulty: Treat as Medium depth (even if tagged Easy/Medium), because it layers “Same Tree” inside a search. [66]
Restatement: Given `root` and `subRoot`, return true if there exists a node in `root` where the subtree matches `subRoot` in structure and values. [67]
What to notice immediately: - This is “find candidate roots” + “verify equality” (repeated concept: Same Tree). [68]
- Worst case: many candidates, repeated comparisons.
- Interview follow-up: hashing/serialization or KMP-on-serialization to optimize (often not required, but good to mention). [16]

B) Brute Force Idea (if applicable)

Brute force: for each node in `root` (O(n)), run `isSameTree(node, subRoot)` (O(m)), total O(n*m) worst case. This is acceptable for many constraints but can be heavy. [68]

C) Optimal Strategy

Pattern: DFS search + SameTree verification. [66]
Invariant: `isSubtree(root, subRoot)` **is true iff either (1)** `root` **matches** `subRoot` **exactly, or (2)** `subRoot` **is a subtree of** `root.left` **or** `root.right`**.**

Key insight: - “Subtree exists somewhere” is exactly the recursive OR decomposition over left/right descendants. [17]

D) Step-by-step Walkthrough (mandatory)

Example:

```
root:        3
            / \
           4   5
          / \
         1   2

subRoot:     4
            / \
           1   2
```

- Check at root=3: sameTree(3,4)? no
- Recurse left: sameTree(4,4)? yes → return true

E) JavaScript Solution (ES2020+)

```
/**
 * Invariant: same(a,b) true iff identical.
```

```
 */
function same(a, b) {
  if (a === null && b === null) return true;
  if (a === null || b === null) return false;
  if (a.val !== b.val) return false;
  return same(a.left, b.left) && same(a.right, b.right);
}

/**
 * Recursive.
 * Invariant: subtree exists iff match at current root OR exists in left OR
exists in right.
 */
function isSubtree(root, subRoot) {
  if (subRoot === null) return true; // empty tree is subtree
  if (root === null) return false;

  if (same(root, subRoot)) return true;
  return isSubtree(root.left, subRoot) || isSubtree(root.right, subRoot);
}

/**
 * Iterative follow-up: DFS over candidate roots; check sameTree at each.
 */
function isSubtreeIterative(root, subRoot) {
  if (subRoot === null) return true;
  if (root === null) return false;

  const stack = [root];

  while (stack.length > 0) {
    const node = stack.pop();
    if (node === null) continue;

    if (same(node, subRoot)) return true;

    if (node.left !== null) stack.push(node.left);
    if (node.right !== null) stack.push(node.right);
  }

  return false;
}
```

F) Complexity

Worst-case time: O(n*m) for naive repeated matching. [68]
Space: O(h) recursion / stack, plus recursion inside `same`. (Depth safety: iterative outer search helps, but

`same` recursion can still overflow if trees are deep; you can also implement `same` iteratively if needed.) [12]

G) Edge Cases & Gotchas - `subRoot = null` → true (by common definition). - `root = null` and `subRoot != null` → false. - Duplicate values in root → many candidates. - Shape must match exactly (not "contains values"). - Extremely deep trees: recursion risks stack overflow. [12]

H) Common Wrong Approaches - "Search for a node with same value as subRoot root and return true": values are insufficient; structure must match. - Serialize without null markers → false positives. [16]

I) Interview Soundbite
"I DFS over the main tree; at each node I check if the subtree rooted there is identical to `subRoot` using the Same Tree helper. If not, I keep searching left and right. That's the clean recursive decomposition; if performance is a concern, I can discuss hashing or robust serialization."

J) Practice Checks 1) Q: Why is Same Tree a natural subroutine here?
A: Subtree match requires identical structure and values at corresponding positions. [69]
2) Q: What's the naive worst-case time?
A: O(n*m) due to repeated comparisons.
3) Q: What serialization pitfall causes false positives?
A: Omitting null children loses shape information. [70]

Gotcha test case: - root = `[1,1,1,1,1]`, subRoot = `[1,1]` with specific shape differences
- Expected: depends on shape; many candidates expose incorrect "value-only" logic.

Lowest Common Ancestor of a Binary Search Tree

A) Problem & Difficulty

Difficulty: Medium. [20]
Restatement: In a BST, find the LCA of nodes `p` and `q`. [20]
What to notice immediately: - BST ordering allows you to "steer" toward both nodes without exploring all nodes. [71]
- If both `p` and `q` are less than current, LCA must be in left subtree; if both greater, in right; otherwise current is split point (the LCA). [72]
- Iterative solution is extremely clean and depth-safe. [73]

B) Brute Force Idea (if applicable)

Brute force: compute path from root to p and root to q (two searches), then scan for last common node. Works but more code and still O(h). BST steering gives direct single-walk solution.

C) Optimal Strategy

Pattern: BST steering (iterative). [72]
Invariant: **At each step, `node` is the root of the smallest subtree that still contains both `p` and `q`.**

Key insight: - The first node where `p` and `q` diverge to different sides (or equal to node) is LCA. [74]

D) Step-by-step Walkthrough (mandatory)

Example BST:

```
        6
       / \
      2   8
     / \ / \
    0  4 7  9
      / \
     3   5
```

Let p=2 and q=8: - At 6: p<6 and q>6 → split → LCA is 6.

Let p=2 and q=4: - At 6: both <6 → go left to 2 - At 2: p==2 → LCA is 2 (a node can be ancestor of itself). [20]

E) JavaScript Solution (ES2020+)

```javascript
/**
 * Iterative (preferred).
 * Invariant: current is the smallest subtree root that still contains both p
and q.
 */
function lowestCommonAncestor(root, p, q) {
  let current = root;
  const pVal = p.val;
  const qVal = q.val;

  // While we haven't found the split point (LCA).
  while (current !== null) {
    if (pVal < current.val && qVal < current.val) {
      current = current.left;   // both targets in left subtree
    } else if (pVal > current.val && qVal > current.val) {
      current = current.right;  // both targets in right subtree
    } else {
      // Split point: one on each side, or current equals one target.
      return current;
    }
  }
```

```
  return null; // if inputs guaranteed present, this line is not reached
}
```

Why `while (current !== null)` is correct: each iteration moves down one level toward the subtree that must contain both nodes; if `current` became null, at least one node wasn't present (outside typical problem guarantees). [20]

F) Complexity

Time: O(h) where h is height of BST (best case balanced: O(log n), worst case skewed: O(n)). [75]
Space: O(1) iterative.

G) Edge Cases & Gotchas - p==q → LCA is p. - One node is ancestor of the other → ancestor is LCA. [20]
- BST with duplicates: LeetCode BST definition typically uses strict inequality; duplicates complicate. [76]
- Skewed BST → O(n) time. - Don't solve as generic binary tree LCA unless asked; BST property gives simpler solution. [77]

H) Common Wrong Approaches - Using generic LCA with full traversal (unnecessary extra work). - Comparing nodes by reference incorrectly if only values are available (in LeetCode, nodes are provided as references).

I) Interview Soundbite
"In a BST, I walk down from the root. If both p and q are less than the current node, I go left; if both are greater, I go right. The first node where they split (or equal the node) is the LCA. This is O(h) time and O(1) space."

J) Practice Checks 1) Q: Why does the split point guarantee lowest common ancestor?
A: Above it, both nodes are in the same subtree; at it, they diverge or match, so no lower node can be ancestor of both.
2) Q: What if p is in left subtree and q is in right subtree of current?
A: current is LCA by definition. [20]
3) Q: Why does this not require recursion?
A: Each step is a deterministic choice; no branching search.

Gotcha test case: - BST root `[2,1,3]`, p=1, q=3 → expected LCA=2.

Binary Tree Level Order Traversal

A) Problem & Difficulty

Difficulty: Medium. [55]
Restatement: Return values level by level as an array of arrays. [55]
What to notice immediately: - It is literally BFS; queue required. [22]
- Must capture level boundaries (`levelSize`) correctly. [54]

B) Brute Force Idea (if applicable)
DFS with depth tracking works, but BFS is simpler and matches the “level-order” requirement directly.

C) Optimal Strategy

Pattern: BFS with queue; snapshot queue length per level. [14]
Invariant: **At the start of each outer loop iteration, the queue contains exactly the nodes of the next unprocessed level.**

Key insight: - After processing `levelSize` nodes, you have completed one level because any newly enqueued children belong to the next level. [54]

D) Step-by-step Walkthrough (mandatory)

Example:

```
     1
    / \
   2   3
  / \
 4   5
```

Queue progression: - Start: [1], levelSize=1 → output [1], enqueue 2,3 - Next: [2,3], levelSize=2 → output [2,3], enqueue 4,5 - Next: [4,5], levelSize=2 → output [4,5]

E) JavaScript Solution (ES2020+)

```javascript
/**
 * BFS.
 * Invariant: queue holds exactly the nodes in the current level before we
process it.
 */
function levelOrder(root) {
  if (root === null) return [];

  const result = [];
  const queue = [root];

  // While there is at least one node in some unprocessed level.
  while (queue.length > 0) {
    const levelSize = queue.length;
    const level = [];

    for (let i = 0; i < levelSize; i++) {
      const node = queue.shift();
```

```
      level.push(node.val);

      if (node.left !== null) queue.push(node.left);
      if (node.right !== null) queue.push(node.right);
    }

    result.push(level);
  }

  return result;
}
```

Why the while-loop condition is correct: BFS proceeds level-by-level; the queue is your frontier; when it empties, there are no more nodes reachable (no more tree nodes). [22]

F) Complexity

Time: O(n). [78]
Space: O(w) ≤ O(n). [11]

G) Edge Cases & Gotchas - Empty tree → `[]`. - Single node → `[[val]]`. - Must snapshot `levelSize` before processing; otherwise the loop length changes mid-level. [54]
- Using `shift()` is O(n) in JS arrays; interviewers may accept, but you can optimize with a head index. - Deep tree is fine; iterative.

H) Common Wrong Approaches - `for (let i = 0; i < queue.length; i++)` directly: queue length mutates. - Enqueueing right before left changes left-to-right ordering requirement.

I) Interview Soundbite
"I use BFS. The queue initially contains the root. For each level, I snapshot the current queue size, pop exactly that many nodes into a level array, and enqueue their children. That guarantees correct level boundaries."

J) Practice Checks 1) Q: Why do we need `levelSize`?
A: Because children enqueued during processing belong to the next level. [54]
2) Q: What's the space bottleneck?
A: Maximum number of nodes at a level (tree width). [79]
3) Q: Can DFS solve it?
A: Yes, collect by depth into arrays, but BFS is the direct expression.

Gotcha test case: - Input: `[1,2,3,4,null,null,5]`
- Expected: `[[1],[2,3],[4,5]]`

Binary Tree Right Side View

A) Problem & Difficulty

Difficulty: Medium. [80]
Restatement: Return the value of the rightmost visible node at each level. [80]
What to notice immediately: - "Rightmost per level" suggests BFS with `levelSize` and pick last node. [81]
- DFS alternative: preorder-like but go right-first; first time hitting a depth is the visible node.

B) Brute Force Idea (if applicable)

Do level-order traversal and pick last element of each level (still O(n)); that's essentially BFS solution.

C) Optimal Strategy

Pattern: BFS level-order; record last node in each level. [82]
Invariant: **After processing a level, the last processed node corresponds to the rightmost node at that level (given left-to-right enqueue).**

Key insight: - If you enqueue left then right, the last node popped at a level is the rightmost node. [83]

D) Step-by-step Walkthrough (mandatory)

Example:

```
    1
   / \
  2   3
   \   \
    5   4
```

Level 0: [1] → rightmost 1
Level 1: [2,3] → rightmost 3
Level 2: [5,4] → rightmost 4
Answer: `[1,3,4]` [84]

E) JavaScript Solution (ES2020+)

```javascript
/**
 * BFS.
 * Invariant: during each level loop, we process exactly that level's nodes.
 * The last processed node at that level is the rightmost.
 */
function rightSideView(root) {
  if (root === null) return [];
```

```
  const result = [];
  const queue = [root];

  while (queue.length > 0) {
    const levelSize = queue.length;
    let rightmost = null;

    for (let i = 0; i < levelSize; i++) {
      const node = queue.shift();
      rightmost = node.val;

      if (node.left !== null) queue.push(node.left);
      if (node.right !== null) queue.push(node.right);
    }

    result.push(rightmost);
  }

  return result;
}

/**
 * DFS alternative (right-first).
 * Invariant: first node encountered at each depth is the rightmost visible
node.
 */
function rightSideViewDFS(root) {
  const result = [];

  function dfs(node, depth) {
    if (node === null) return;

    if (depth === result.length) {
      result.push(node.val);
    }

    dfs(node.right, depth + 1);
    dfs(node.left, depth + 1);
  }

  dfs(root, 0);
  return result;
}
```

F) Complexity

Time: O(n). [85]
Space: O(w) queue or O(h) recursion. [37]

G) Edge Cases & Gotchas - Empty tree → `[]`. - Single node → `[val]`. - If you enqueue right before left in BFS, "last node" flips meaning; be consistent. - DFS depth-first: must traverse right child first. - Deep recursion in JS: BFS preferred. [12]

H) Common Wrong Approaches - Returning all right children only (misses cases where right child absent but left exists). - BFS without level boundary: you might push wrong results.

I) Interview Soundbite
"I do BFS by levels. For each level, I process exactly the current queue size nodes and keep overwriting `rightmost`; the final value is the rightmost node at that level. Alternatively, a right-first DFS records the first node seen at each depth."

J) Practice Checks 1) Q: Why does right-first DFS work?
A: The first node visited at depth d is the rightmost if we prioritize right edges.
2) Q: What BFS ordering ensures the last popped per level is rightmost?
A: Enqueue left then right.
3) Q: Which approach avoids recursion depth risk?
A: BFS.

Gotcha test case: - Input: `[1,2,3,null,5,null,4]`
- Expected: `[1,3,4]` [86]

Count Good Nodes in Binary Tree

A) Problem & Difficulty

Difficulty: Medium. [87]
Restatement: A node is "good" if on the path from root to that node, no ancestor has a value greater than the node (equivalently, node value is ≥ max-so-far along the path). Return count of good nodes. [88]
What to notice immediately: - This is root-to-node path state; carry `maxSoFar`. - Comparison is ≥, not `>` (common gotcha). [89]
- DFS is natural; BFS also works.

B) Brute Force Idea (if applicable)
For each node, walk its ancestor path to check max (O(h)) → O(n*h) worst case O(n^2). Better: maintain max along traversal.

C) Optimal Strategy

Pattern: DFS with path accumulator (max so far). [88]
Invariant: `dfs(node, maxSoFar)` **visits each node with** `maxSoFar` **equal to the maximum value on the path from root to** `node` **(inclusive of ancestors).**

Key insight: - Update max as you go down; count if `node.val >= maxSoFar`.

D) Step-by-step Walkthrough (mandatory)

Example:

```
    3
   / \
  1   4
     / \
    1   5
```

Start at 3: max=3 → good (3>=3)
Go left 1: max=3 → not good
Go right 4: max=3 → good (4>=3), new max=4
Then 1: max=4 → not good
Then 5: max=4 → good

Total good = 3.

E) JavaScript Solution (ES2020+)

```javascript
/**
 * Recursive.
 * Invariant: maxSoFar is the max on the path from root to current node's parent
(or root).
 */
function goodNodes(root) {
  function dfs(node, maxSoFar) {
    if (node === null) return 0;

    const isGood = node.val >= maxSoFar ? 1 : 0;
    const newMax = Math.max(maxSoFar, node.val);

    return isGood + dfs(node.left, newMax) + dfs(node.right, newMax);
  }

  if (root === null) return 0;
  return dfs(root, root.val);
}

/**
 * Iterative follow-up (depth-safe).
 * Invariant: stack holds pairs (node, maxSoFarAlongPathToNode).
 */
```

```
function goodNodesIterative(root) {
  if (root === null) return 0;
  let count = 0;
  const stack = [[root, root.val]];

  while (stack.length > 0) {
    const [node, maxSoFar] = stack.pop();
    if (node.val >= maxSoFar) count += 1;
    const newMax = Math.max(maxSoFar, node.val);

    if (node.left !== null) stack.push([node.left, newMax]);
    if (node.right !== null) stack.push([node.right, newMax]);
  }

  return count;
}
```

F) Complexity

Time: O(n). [88]
Space: O(h) recursion or O(h) stack. [45]

G) Edge Cases & Gotchas - Empty tree → 0. - All equal values → all nodes are good (needs ≥). [89]
- Negative values: initialize max carefully (using root value is fine). - Long chain with many equal nodes: ensure ≥ not >. - Deep recursion risk in JS. [12]

H) Common Wrong Approaches - Using `>` and excluding equal-max nodes (wrong per definition). [89]
- Resetting max at each subtree root (breaks "path" concept).

I) Interview Soundbite
"I DFS while carrying the maximum value seen so far on the path. A node is good if its value is at least that max. I update the max when descending. This visits each node once, O(n), and the iterative version uses an explicit stack."

J) Practice Checks 1) Q: Why do we carry max instead of recomputing?
A: To avoid per-node ancestor scans; it makes it one-pass O(n).
2) Q: What's the comparison operator?
A: `>=`, because ancestors must not be greater; equality is allowed. [89]
3) Q: What state do you store in an iterative stack?
A: `(node, maxSoFar)`.

Gotcha test case: - Input: `[2,2,2,2,null,2]`
- Expected: all nodes count as good.

Validate Binary Search Tree

A) Problem & Difficulty

Difficulty: Medium. [76]
Restatement: Determine if a binary tree is a valid BST: all left subtree values strictly less than node, all right subtree values strictly greater, recursively. [90]
What to notice immediately: - Must enforce **global** constraints (range propagation), not local comparisons. [91]
- Strict inequality matters (`<` and `>`). [90]
- Iterative solution: stack of `(node, min, max)` avoids recursion depth risk. [12]

B) Brute Force Idea (if applicable)

Brute-ish: inorder traversal then check strictly increasing sequence (O(n) time), but requires storing order or tracking prev and only works if duplicates are handled consistently. Range propagation is more directly aligned to definition. [91]

C) Optimal Strategy

Pattern: DFS with `(low, high)` bounds. [91]
Invariant: `dfs(node, low, high)` **is true iff every value in** `node`**'s subtree lies strictly between** `(low, high)` **and both child subtrees satisfy their tightened bounds.**

Key insight: - On left: new high becomes `node.val`; on right: new low becomes `node.val`. [92]

D) Step-by-step Walkthrough (mandatory)

Classic failing case:

```
    5
   / \
  4   6
     / \
    3   7
```

Local checks might pass at node 6 (3<6<7), but 3 is in the right subtree of 5, violating global ">5" constraint. Range approach catches it: - root 5: range (-inf, +inf) - right child 6: range (5, +inf) - left child of 6 is 3: must be in (5,6) → fails.

E) JavaScript Solution (ES2020+)

```
/**
 * Recursive range propagation.
```

```
 * Invariant: node values must lie within (low, high) exclusively.
 */
function isValidBST(root) {
  function dfs(node, low, high) {
    if (node === null) return true;

    if (!(low < node.val && node.val < high)) return false;

    return dfs(node.left, low, node.val) && dfs(node.right, node.val, high);
  }

  return dfs(root, -Infinity, Infinity);
}

/**
 * Iterative depth-safe follow-up.
 * Invariant: stack holds (node, low, high) constraints that must be satisfied.
 */
function isValidBSTIterative(root) {
  const stack = [[root, -Infinity, Infinity]];

  while (stack.length > 0) {
    const [node, low, high] = stack.pop();
    if (node === null) continue;

    if (!(low < node.val && node.val < high)) return false;

    stack.push([node.right, node.val, high]);
    stack.push([node.left, low, node.val]);
  }

  return true;
}
```

Why `while (stack.length > 0)` is correct: each triple represents an outstanding subtree constraint check; empty stack means every subtree was checked with valid bounds.

F) Complexity

Time: O(n). [8]
Space: O(h) recursion/stack. [45]

G) Edge Cases & Gotchas - Empty tree → true. - Single node → true. - Duplicates: strict BST definition usually disallows equal values in wrong side; use strict inequality. [90]
- Don't use `Number.MIN_VALUE` (that's smallest positive), use `-Infinity`. - Deep tree: prefer iterative. [12]

H) Common Wrong Approaches - Only checking `node.left.val < node.val < node.right.val` (misses deep violations). [92]
- Inorder approach but allowing equals incorrectly, or not handling integer edge bounds.

I) Interview Soundbite
"I validate a BST by propagating allowed ranges. Each node must be strictly between `(low, high)`. Left subtree tightens the upper bound to `node.val`, right subtree tightens the lower bound. This catches deep violations that local checks miss, and the iterative version stores `(node, low, high)` on a stack."

J) Practice Checks 1) Q: Why are local comparisons insufficient?
A: Because a node can violate an ancestor constraint even if it's locally ordered with its parent. [92]
2) Q: What inequality do we use and why?
A: Strict (`<`), because BST definition requires all left strictly less and all right strictly greater. [76]
3) Q: What's the iterative state?
A: Node plus its valid range bounds.

Gotcha test case: - Input: `[5,4,6,null,null,3,7]`
- Expected: `false` [93]

Kth Smallest Element in a BST

A) Problem & Difficulty

Difficulty: Medium. [94]
Restatement: Return the k-th smallest value (1-indexed) in a BST. [94]
What to notice immediately: - Inorder traversal of BST yields sorted order. [95]
- Iterative inorder is interview-favorite (controls recursion depth). [12]

B) Brute Force Idea (if applicable)

Brute force: inorder collect all values, sort (O(n log n)) or rely on inorder then index (O(n) memory). Optimal: stop early after k steps.

C) Optimal Strategy

Pattern: Inorder traversal, counting visited nodes. [95]
Invariant: **During inorder, the sequence of visited node values is strictly increasing in a valid BST, so the k-th visit is the answer.**

D) Step-by-step Walkthrough (mandatory)

Example BST:

```
    3
   / \
  1   4
   \
    2
```

Inorder: 1,2,3,4. If k=2 → answer 2.

Iterative inorder stack trace: - push left chain: 3→1 - pop 1 (count=1), go right to 2 - push 2, pop 2 (count=2) → return 2

E) JavaScript Solution (ES2020+)

```javascript
/**
 * Iterative inorder (preferred).
 * Invariant: stack contains path to next unvisited node, maintaining inorder
order.
 */
function kthSmallest(root, k) {
  const stack = [];
  let current = root;
  let count = 0;

  // While there are nodes not yet processed: either in the stack or via current
pointer.
  while (current !== null || stack.length > 0) {
    // Go as left as possible, pushing ancestors.
    while (current !== null) {
      stack.push(current);
      current = current.left;
    }

    const node = stack.pop();
    count += 1;
    if (count === k) return node.val;

    // After visiting node, traverse its right subtree.
    current = node.right;
  }

  return null; // if k is valid per problem constraints, not reached
}
```

Explain the while conditions: - Outer `while (current !== null || stack.length > 0)` means: there exists a next node in inorder either because we are mid-descent (`current`) or we have ancestors

waiting (`stack`). - Inner `while (current !== null)` pushes the entire left spine so that the next pop is the leftmost unvisited node—exactly inorder.

F) Complexity

Time: O(h + k) average; O(n) worst case if k ~ n. [96]
Space: O(h) stack. [45]

G) Edge Cases & Gotchas - k=1 → minimum element (leftmost). - k=n → maximum element. - Skewed BST → O(n) height. - Must be 1-indexed (problem statement). [94]
- Assumes BST is valid; not asked to validate simultaneously.

H) Common Wrong Approaches - Doing BFS and sorting values: unnecessary. - Returning after pushing k nodes instead of popping k nodes (order mistake).

I) Interview Soundbite
"I use iterative inorder traversal because inorder visits BST values in sorted order. I keep a counter incremented when I pop a node (the moment it's 'visited'). When the counter reaches k, that node is the k-th smallest."

J) Practice Checks 1) Q: Why increment on pop, not push?
A: Pop corresponds to inorder visitation (after fully processing left subtree).
2) Q: What is the next subtree after visiting a node?
A: The node's right subtree.
3) Q: What's the maximum stack size?
A: Height of the tree.

Gotcha test case: - Input: `[1,null,2,null,3]`, k=2
- Expected: `2`

Construct Binary Tree From Preorder And Inorder Traversal

A) Problem & Difficulty

Difficulty: Medium. [97]
Restatement: Given preorder and inorder arrays representing the same tree, reconstruct and return the tree. [97]
What to notice immediately: - Preorder gives root first. - In inorder, root splits left and right subtrees. [98]
- Need an index map for inorder positions to achieve O(n). [99]

B) Brute Force Idea (if applicable)

Brute: find root index in inorder by scanning each time → O(n^2) worst case.

C) Optimal Strategy

Pattern: Divide-and-conquer recursion with inorder index map. [99]
Invariant: `build(preL, preR, inL, inR)` **returns the exact subtree formed by those slices, preserving traversal definitions.**

Key insight: - Root is `preorder[preL]`; find its inorder position to determine left subtree size.

D) Step-by-step Walkthrough (mandatory)

Example: - preorder = [3,9,20,15,7] - inorder = [9,3,15,20,7]

Root = 3 (first preorder). In inorder, 3 is at index 1: - left inorder = [9] (size 1) - right inorder = [15,20,7]

So left preorder slice after root of size 1 is [9]. Right preorder is [20,15,7]. Recurse similarly.

E) JavaScript Solution (ES2020+)

```
/**
 * Recursive with hashes.
 * Invariant: build(preL, preR, inL, inR) constructs subtree from those ranges
inclusive.
 */
function buildTree(preorder, inorder) {
  const index = new Map();
  for (let i = 0; i < inorder.length; i++) index.set(inorder[i], i);

  function build(preL, preR, inL, inR) {
    if (preL > preR) return null;

    const rootVal = preorder[preL];
    const rootInIdx = index.get(rootVal);
    const leftSize = rootInIdx - inL;

    const root = new TreeNode(rootVal);
    root.left = build(preL + 1, preL + leftSize, inL, rootInIdx - 1);
    root.right = build(preL + leftSize + 1, preR, rootInIdx + 1, inR);
    return root;
  }

  return build(0, preorder.length - 1, 0, inorder.length - 1);
}
```

Depth-safe follow-up note: recursion depth is O(h) and could overflow if the tree is skewed (e.g., strictly ascending inorder with preorder matching). In JS, an interviewer may accept recursion given constraints, but you should mention using an iterative stack construction if necessary. [100]

F) Complexity

Time: O(n) with index map. [101]
Space: O(n) for the map + O(h) recursion stack. [102]

G) Edge Cases & Gotchas - Empty arrays → null. - Single element arrays. - Values assumed unique in typical statement (needed for index map uniqueness). [97]
- Off-by-one in slice boundaries. - Skewed tree → deep recursion. [12]

H) Common Wrong Approaches - Scanning inorder for root each time → O(n^2). - Incorrectly computing left subtree size, causing mis-sliced preorder.

I) Interview Soundbite
"Preorder gives me the root immediately; inorder tells me how to split left and right subtrees around that root. With a hash map from value to inorder index, I can compute subtree sizes and recursively build left and right ranges in O(n) time."

J) Practice Checks 1) Q: Why do we need unique values?
A: Otherwise "the" inorder index of root is ambiguous.
2) Q: What determines the size of the left subtree?
A: Distance from `inL` to the root's inorder index.
3) Q: What's the main bug source?
A: Off-by-one errors in index ranges.

Gotcha test case: - preorder `[1,2,3,4]`, inorder `[1,2,3,4]` (skewed right) → deep recursion potential.

Binary Tree Maximum Path Sum

A) Problem & Difficulty

Difficulty: Hard. [103]
Restatement: Return the maximum sum of any non-empty path, where a path is any sequence of connected nodes (can start and end anywhere) and doesn't need to pass through root. [34]
What to notice immediately: - Two different "path" notions: 1) best **downward** path from node to any descendant (returned to parent), 2) best **anywhere** path (global best, may pass through node using both children). [104]
- Negative values mean you may want to drop (clamp) a child contribution to 0. [105]

B) Brute Force Idea (if applicable)

Brute force: enumerate all pairs of nodes and compute path sums → too slow; trees have O(n^2) pairs; computing paths naïvely is heavy. DP/postorder is intended.

C) Optimal Strategy

Pattern: Postorder DFS returning max downward gain; update global best with “through node” path. [104]
Invariant: `dfs(node)` **returns the maximum sum of a path that starts at** `node` **and goes downward to one side (left or right), and** `best` **is the maximum path sum anywhere in the processed subtrees.**

Key insight: - At node, best path using node as highest point is `node.val + max(0,leftGain) + max(0,rightGain)`. [105]

D) Step-by-step Walkthrough (mandatory)

Example (classic):

```
      -10
      /  \
     9    20
         /  \
        15   7
```

Postorder compute gains: - dfs(9) = 9, best = 9 - dfs(15) = 15, best = 15 - dfs(7) = 7, best = 15 - at 20: - leftGain=15, rightGain=7 - through = 20+15+7 = 42 → best=42 - return downward gain = 20 + max(15,7) = 35 - at -10: - leftGain=9, rightGain=35 - through = -10 + 9 + 35 = 34, best stays 42 - return downward gain = -10 + max(9,35) = 25

Answer: 42. [106]

E) JavaScript Solution (ES2020+)

```javascript
/**
 * Recursive postorder.
 * Invariant: dfs(node) returns max downward gain starting at node (one-branch path).
 * best tracks max path sum anywhere.
 */
function maxPathSum(root) {
  let best = -Infinity;

  function dfs(node) {
    if (node === null) return 0;

    const leftGain = Math.max(0, dfs(node.left));
    const rightGain = Math.max(0, dfs(node.right));

    // Best path that uses node as the "peak" (can take both sides).
    best = Math.max(best, node.val + leftGain + rightGain);

    // Return best downward path (must choose one side).
```

```
    return node.val + Math.max(leftGain, rightGain);
  }

  dfs(root);
  return best;
}

/**
 * Iterative depth-safe follow-up (postorder with visited flag + Map of gains).
 * Invariant: when node is processed (visited=true), its children gains are
known.
 */
function maxPathSumIterative(root) {
  if (root === null) return -Infinity;

  let best = -Infinity;
  const gain = new Map();
  const stack = [[root, false]];

  while (stack.length > 0) {
    const [node, visited] = stack.pop();
    if (node === null) continue;

    if (!visited) {
      stack.push([node, true]);
      stack.push([node.right, false]);
      stack.push([node.left, false]);
    } else {
      const leftGain = node.left ? Math.max(0, gain.get(node.left)) : 0;
      const rightGain = node.right ? Math.max(0, gain.get(node.right)) : 0;

      best = Math.max(best, node.val + leftGain + rightGain);
      gain.set(node, node.val + Math.max(leftGain, rightGain));
    }
  }

  return best;
}
```

F) Complexity

Time: O(n). [34]
Space: O(h) recursion or O(n) Map+stack iterative. [102]

G) Edge Cases & Gotchas - All negative values: answer is the maximum single node (don't clamp everything away globally). This is why `best` starts at `-Infinity`. [107]

- Single node tree. - Mixed positive/negative. - Confusing “downward gain” with “through node” path. - Deep recursion risk in JS. [12]

H) Common Wrong Approaches - Returning `node.val + leftGain + rightGain` to parent (illegal: parent path would fork). - Initializing `best` to 0 (fails when all values negative). [103]

I) Interview Soundbite
“I do a postorder traversal. Each node returns the best downward gain—node value plus the max of left or right gains, clamped at 0 so we ignore negative branches. Separately, I update a global best with a path that uses the node as a peak: node plus both left and right gains. That handles paths that start and end anywhere.”

J) Practice Checks 1) Q: Why can downward gain only pick one side?
A: A path to the parent cannot branch; it must be a simple path. [103]
2) Q: Why clamp child gains at 0?
A: Negative subpaths reduce sum; excluding them increases or preserves optimality. [108]
3) Q: Why does `best` start at -Infinity?
A: Because the maximum path must be non-empty and could be negative.

Gotcha test case: - Input: `[-3]`
- Expected: `-3`

Serialize And Deserialize Binary Tree

A) Problem & Difficulty

Difficulty: Hard. [70]
Restatement: Design an algorithm to serialize a binary tree to a string, and deserialize it back to the identical tree. No restriction on format. [16]
What to notice immediately: - You are defining a contract; must preserve shape, so you need explicit null markers or equivalent. [16]
- Two common approaches: BFS (level-order) with “null” tokens, or DFS preorder with “#” for null. - Interview follow-up: confirm that serializer + deserializer are inverses.

B) Brute Force Idea (if applicable)
Not relevant: any working serialization format is acceptable, but correctness and unambiguity are key.

C) Optimal Strategy

Pattern: BFS serialization with null sentinels; BFS deserialization reconstructs children in order. [109]
Invariant: **During deserialization, the queue contains parents whose children positions (left then right) are the next tokens to be consumed.**

Key insight: - Without null markers, trees with different shapes can collide in serialization. [16]

D) Step-by-step Walkthrough (mandatory)

Tree:

```
    1
   / \
  2   3
     /
    4
```

BFS tokens with null markers (one possible format): `1,2,3,null,null,4,null,null,null`

Deserialization: - Create root=1, queue=[1] - Pop 1: next tokens are 2 and 3 → attach left=2 right=3; queue=[2,3] - Pop 2: next tokens null null → children empty - Pop 3: next tokens 4 null → left=4; queue=[4] - Pop 4: next tokens null null → done once tokens consumed.

E) JavaScript Solution (ES2020+)

```javascript
/**
 * BFS serializer/deserializer using "null" tokens.
 * Note: Format is arbitrary; must be consistent across both functions.
 */
function serialize(root) {
  if (root === null) return "";

  const queue = [root];
  const tokens = [];

  // While there are nodes whose children positions haven't been encoded.
  while (queue.length > 0) {
    const node = queue.shift();
    if (node === null) {
      tokens.push("null");
      continue;
    }

    tokens.push(String(node.val));
    queue.push(node.left);
    queue.push(node.right);
  }

  // Optional: trim trailing "null" tokens to shorten output (must handle in
deserialize if you do).
  return tokens.join(",");
}
```

```
function deserialize(data) {
  if (data === "") return null;
  const tokens = data.split(",");
  const rootVal = tokens[0];
  const root = new TreeNode(Number(rootVal));

  const queue = [root];
  let i = 1;

  // While there are parent nodes waiting for children AND tokens remain.
  while (queue.length > 0 && i < tokens.length) {
    const parent = queue.shift();

    const leftTok = tokens[i++];
    if (leftTok !== "null") {
      parent.left = new TreeNode(Number(leftTok));
      queue.push(parent.left);
    }

    if (i >= tokens.length) break;

    const rightTok = tokens[i++];
    if (rightTok !== "null") {
      parent.right = new TreeNode(Number(rightTok));
      queue.push(parent.right);
    }
  }

  return root;
}
```

Why the while-loop condition is correct: - `queue.length > 0` means there exists at least one parent node whose child pointers still need to be assigned. - `i < tokens.length` means there still exist tokens representing child positions to consume.

F) Complexity

Time: O(n) serialize and deserialize. [110]
Space: O(n) for tokens and BFS queue. [111]

G) Edge Cases & Gotchas - Empty tree: return `""` and deserialize to null. - Single node: `"1"` round-trips. - Trees with missing internal children must encode nulls, or shape is lost. [16]
- Be explicit about token parsing: numbers vs strings. - Trailing null trimming must be mirrored by the parser; otherwise index logic breaks.

H) Common Wrong Approaches - Omitting nulls: ambiguous reconstruction. [16]
- Using DFS serialize but BFS deserialize (contracts mismatch).

I) Interview Soundbite
"I pick a format and ensure it's unambiguous. A simple approach is BFS level-order with explicit `null` tokens for missing children. Serialization is a BFS that outputs node values or `null`; deserialization consumes tokens in order, using a queue of parents and assigning left then right children. Both run in O(n)."

J) Practice Checks 1) Q: Why do we need null tokens?
A: Without them, different tree shapes can serialize to identical sequences. [70]
2) Q: What invariant does the deserialization queue maintain?
A: It stores parents whose children are the next tokens to consume.
3) Q: If you trim trailing nulls in serialize, what must change?
A: Deserialize must work even when final null-child positions are omitted.

Gotcha test case: - Input tree: `[1,2,3,null,null,4,null]`
- Expected: `deserialize(serialize(tree))` reconstructs same structure; right child of 3 is null, left child is 4.

Tree Interview Playbook

A seven-step checklist for solving any tree problem

1. **Restate precisely**: clarify whether answers are in nodes vs edges; confirm "path" definition and whether it must include root. [15]
2. **Pick traversal**:
3. preorder for transformations,
4. postorder for child-derived computations,
5. inorder for BST sorted properties,
6. BFS for per-level outputs. [23]
7. **Define helper return meaning** in one sentence (height/boolean/gain/tuple).
8. **Write base cases upfront**, especially `null`.
9. **Write the invariant sentence** and ensure each line of code preserves it.
10. **Prove complexity**: "each node visited once" → O(n); explain stack/queue size in terms of height/width. [65]
11. **Prepare depth-safe follow-up**: if recursion depth can be large, convert to iterative using an explicit stack/queue; JS can throw stack overflow errors on deep recursion. [12]

How to choose traversal quickly

- "Compute something from children" → postorder. Examples: depth, diameter, balance, max path sum. [112]
- "Return by levels / right side view" → BFS. [113]
- "BST k-th / sorted order" → inorder. [95]
- "Transform/copy tree" → preorder (invert). [114]

How to decide what recursion should return

Ask: *What does the parent need from the child?* - Parent needs child height → return height. - Parent needs to know validity → return boolean (or sentinel). - Parent needs best downward contribution → return gain. - Parent needs both “best so far” and “height” → return height and update global best.

This decision is the difference between a clean solution and a tangled one.

Common patterns mapping

Pattern	Tells	Return value	Typical problems in this set
Preorder transform	“Modify tree in place,” “mirror,” “copy/transform”	node (or nothing)	Invert Binary Tree
Postorder height DP	“depth/height,” “balanced,” “diameter”	height (or sentinel)	Max Depth, Balanced, Diameter
Postorder with global best	“maximum path,” “diameter,” “best anywhere”	downward gain + global update	Diameter, Max Path Sum
Inorder counting	“k-th smallest,” “sorted in BST”	none (counter)	Kth Smallest
Range propagation	“validate BST”	boolean	Validate BST
BFS level boundary	“level order,” “right side view”	arrays / rightmost	Level Order, Right Side View
Contracted encoding	“serialize/deserialize”	string ↔ tree	Serialize/Deserialize

This table is the “mental index” you should recall mid-interview: recognize the tell, pick the pattern, state the invariant, and implement cleanly.

---

1 4 5 6 17 18 43 Tree (abstract data type)
https://en.wikipedia.org/wiki/Tree_%28abstract_data_type%29?utm_source=chatgpt.com

2 12 100 102 InternalError: too much recursion - JavaScript | MDN - Mozilla
https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Errors/Too_much_recursion?utm_source=chatgpt.com

3 94 95 96 Kth Smallest Element in a BST
https://leetcode.com/problems/kth-smallest-element-in-a-bst/?utm_source=chatgpt.com

7 9 13 21 23 44 47 50 51 7.2 Binary Tree Traversal - Hello Algo
https://www.hello-algo.com/en/chapter_tree/binary_tree_traversal/?utm_source=chatgpt.com

8 36 76 90 98. Validate Binary Search Tree
https://leetcode.com/problems/validate-binary-search-tree/?utm_source=chatgpt.com

10 11 22 29 37 79 111 Breadth-first search
https://en.wikipedia.org/wiki/Breadth-first_search?utm_source=chatgpt.com

14 55 78 113 102. Binary Tree Level Order Traversal
https://leetcode.com/problems/binary-tree-level-order-traversal/?utm_source=chatgpt.com

15 24 56 57 543. Diameter of Binary Tree
https://leetcode.com/problems/diameter-of-binary-tree/?utm_source=chatgpt.com

16 70 110 Serialize and Deserialize Binary Tree
https://leetcode.com/problems/serialize-and-deserialize-binary-tree/?utm_source=chatgpt.com

19 45 49 65 75 Binary Trees
https://www.andrew.cmu.edu/course/15-121/lectures/Trees/trees.html?utm_source=chatgpt.com

20 27 39 74 Lowest Common Ancestor of a Binary Search Tree
https://leetcode.com/problems/lowest-common-ancestor-of-a-binary-search-tree/?utm_source=chatgpt.com

25 62 Balanced Binary Tree
https://leetcode.com/problems/balanced-binary-tree/?utm_source=chatgpt.com

26 Document Object Model (DOM) - Web APIs | MDN - Mozilla
https://developer.mozilla.org/en-US/docs/Web/API/Document_Object_Model?utm_source=chatgpt.com

28 Understanding Your UI as a Tree
https://react.dev/learn/understanding-your-ui-as-a-tree?utm_source=chatgpt.com

30 Trie
https://en.wikipedia.org/wiki/Trie?utm_source=chatgpt.com

31 Abstract syntax tree
https://en.wikipedia.org/wiki/Abstract_syntax_tree?utm_source=chatgpt.com

32 53 54 102. Binary Tree Level Order Traversal
https://algo.monster/liteproblems/102?utm_source=chatgpt.com

33 Eloquent: Relationships - Laravel 12.x - The PHP Framework For Web Artisans
https://laravel.com/docs/12.x/eloquent-relationships

34 103 107 Binary Tree Maximum Path Sum
https://leetcode.com/problems/binary-tree-maximum-path-sum/?utm_source=chatgpt.com

35 48 52 112 Maximum Depth of Binary Tree
https://leetcode.com/problems/maximum-depth-of-binary-tree/?utm_source=chatgpt.com

38 91 92 Check if a Binary Tree is BST or not
https://www.geeksforgeeks.org/dsa/a-program-to-check-if-a-binary-tree-is-bst-or-not/?utm_source=chatgpt.com

40 NeetCode 150 - Coding Interview Questions
https://neetcode.io/practice/practice/neetcode150?utm_source=chatgpt.com

41 42 46 114 Invert Binary Tree
https://leetcode.com/problems/invert-binary-tree/?utm_source=chatgpt.com

58 59 60 61 Diameter of a Binary Tree
https://www.geeksforgeeks.org/dsa/diameter-of-a-binary-tree/?utm_source=chatgpt.com

[63] [64] [69] Same Tree
https://leetcode.com/problems/same-tree/?utm_source=chatgpt.com

[66] [67] [68] Subtree of Another Tree
https://leetcode.com/problems/subtree-of-another-tree/?utm_source=chatgpt.com

[71] [72] [73] [77] 235. Lowest Common Ancestor of a Binary Search Tree
https://neetcode.io/solutions/lowest-common-ancestor-of-a-binary-search-tree?utm_source=chatgpt.com

[80] [81] [82] [84] [85] Binary Tree Right Side View
https://leetcode.com/problems/binary-tree-right-side-view/?utm_source=chatgpt.com

[83] 199. Binary Tree Right Side View - LeetCode Solutions
https://walkccc.me/LeetCode/problems/199/?utm_source=chatgpt.com

[86] LeetCode 199. Binary Tree Right Side View— Python ...
https://medium.com/codex/leetcode-199-binary-tree-right-side-view-python-programming-solution-678e64a05512?utm_source=chatgpt.com

[87] [88] Count Good Nodes in Binary Tree
https://leetcode.com/problems/count-good-nodes-in-binary-tree/?utm_source=chatgpt.com

[89] 1448. Count Good Nodes in Binary Tree
https://algo.monster/liteproblems/1448?utm_source=chatgpt.com

[93] LeetCode 98: Validate Binary Search Tree
https://stackoverflow.com/questions/73232651/leetcode-98-validate-binary-search-tree?utm_source=chatgpt.com

[97] [98] Construct Binary Tree from Preorder and Inorder Traversal
https://leetcode.com/problems/construct-binary-tree-from-preorder-and-inorder-traversal/?utm_source=chatgpt.com

[99] 105. Construct Binary Tree from Preorder and Inorder ...
https://algo.monster/liteproblems/105?utm_source=chatgpt.com

[101] 105. Construct Binary Tree from Preorder and Inorder ...
https://walkccc.me/LeetCode/problems/105/?utm_source=chatgpt.com

[104] [105] [108] 124. Binary Tree Maximum Path Sum - Explanation
https://neetcode.io/solutions/binary-tree-maximum-path-sum?utm_source=chatgpt.com

[106] 124. Binary Tree Maximum Path Sum - In-Depth Explanation
https://algo.monster/liteproblems/124?utm_source=chatgpt.com

[109] 297. Serialize and Deserialize Binary Tree - LeetCode ...
https://walkccc.me/LeetCode/problems/297/?utm_source=chatgpt.com